BHTTP/1 — HTTP semantics in binary frames

Version 1.0. The key words MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. All integers are unsigned and big-endian (network order). "Octet" means 8 bits.

1. Model

A client opens **one** TCP connection and sends requests over it. The server answers each request **in the order received**, and the connection stays open after every response until one side closes it. A client MAY pipeline (send request *n+1* before response *n* arrives). A server MAY close a connection that has been idle between frames (bserve: 60 s).

2. Connection preface

Immediately after connecting, **each side** sends 4 octets, without waiting for the other:

```
42 48  MAJOR  MINOR      "BH", then the version. Version 1.0 = 42 48 01 00
```

- Magic is not `42 48` → the server sends a `400` response on req_id 0 (§3) and closes. A client reports "not a BHTTP server".
- MAJOR ≠ 1 → the server sends `505` on req_id 0 and closes.
- MINOR is ignored. A minor revision may only *add* things a v1.0 peer is required to skip (§4, §5).

3. Frame header (8 octets)

Everything after the preface is a sequence of frames:

```
 0               1               2               3
 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7
+---------------------------------------------------------------+
|                          Length (32)                          |
+---------------+---------------+-------------------------------+
|   Type (8)    |   Flags (8)   |         Req ID (16)           |
+---------------+---------------+-------------------------------+
|                 Payload (Length octets) ...                   |
```

- **Length** is the number of payload octets, not counting the 8-octet header. Range 0 to $2^{32}-1$.
- **Type** says what the payload is (§4).
- **Flags** is a bit field. v1 defines one bit, `0x01 END_STREAM`, which marks the last frame of a message. Senders MUST send undefined bits as 0, and receivers MUST ignore them.
- **Req ID** is chosen by the client for each request. The server copies it into every frame of the matching response. A client SHOULD number requests 1, 2, 3, … and wrap after 65535, skipping 0. **0 is reserved** for connection-level replies from the server (bad preface, §2). A request that uses 0 gets a `400` reply.

4. Frame types

`0x00` **DATA**: the payload is message body bytes (opaque). `0x01` **HEADERS**: the payload is a header block (§5). Every other type is **unknown**.

A receiver meeting a frame type it does not know MUST skip it cleanly. It reads and discards exactly *Length* payload octets and carries on. This holds in every state, whatever the frame's flags or req_id. This rule is the extension point for later versions: a v2 can add frame types (PING, GOAWAY, PRIORITY, …) and a v1 peer will step over them.

A **message** (request or response) is exactly one HEADERS frame followed by zero or more DATA frames. The frame that carries END_STREAM ends the message, so a HEADERS frame with END_STREAM set means there is no body. A sender MAY split a body at any byte boundary, including into empty DATA frames, and a receiver MUST join the pieces back together. Unknown frames MAY appear between any two frames.

5. Header block

A HEADERS payload is a run of fields packed back to back until the payload ends. There is no field count. Each field is:

```
+-------+ - - - - - - - - - - - - - -+---------------+--------------------+
| Index | [NameLen (8) | Name octets]  | ValueLen (16) | Value octets       |
+-------+ - - - - - - - - - - - - - -+---------------+--------------------+
          present only when Index = 0
```

Index (1 octet): `1–10` picks a name from the static table below. `0` means a literal name follows, as NameLen (1–255) and then the name octets. `11–255` are **reserved**: the receiver MUST still read ValueLen and the value, and then MUST ignore the field. (This is how a minor version can add table entries.)

Sent by	Static table
client	`1` :method · `2` :path · `3` host · `4` user-agent · `5` accept
server	`6` :status · `7` content-type · `8` content-length · `9` server · `10` date

These are exactly the ten names our two programs send on every message. Any other name, such as `allow` on a 405 or one added with `bcurl -H`, goes as a literal. A sender SHOULD use the index whenever the name is in the table. A receiver MUST also accept a table name sent as a literal.

Names MUST be lowercase HTTP token characters: `a–z`, `0–9`, `! # $ % & ' * + - . ^ _ | ~` and the backtick. The one exception is a leading `:`, which marks a pseudo-header. **Values** are 0–65535 octets and MUST NOT contain NUL, CR or LF. They are text, with HTTP/1.1 meanings: `:status` is three ASCII digits and `content-length` is ASCII decimal.

A block is **malformed** if any length runs past the end of the payload, if NameLen is 0, or if a name or value contains a forbidden octet.

6. Requests, responses, errors

Request: exactly one `:method` and exactly one `:path`. `:path` starts with `/`, is percent-encoded and may carry `?query`. No other pseudo-header is allowed. `host` SHOULD be sent. v1 servers implement `GET` and `HEAD`. A request MAY carry a body: HEADERS without END_STREAM, then DATA frames. A v1 server reads that body and discards it before it replies.

Response: one `:status` (`100–599`). The server SHOULD send `content-length`. If it is present, the client MUST check it against the body it received and treat a mismatch as an error. A response to `HEAD` has no DATA frames.

Errors. The rule is: if the frame boundaries can still be trusted, only the request fails. If they can't, the connection fails.

Condition	Reply	Connection
file missing / outside root / method not GET or HEAD	`404` / `403` / `405` + `allow`	stays open
malformed header block; missing or duplicate `:method`/`:path`; pseudo-header not allowed in a request; req_id 0; bad `:path`	`400`	**stays open**
bad preface / unsupported major version	`400` / `505` on req_id 0	closed
DATA when no request is open; HEADERS while a request body is still open; HEADERS longer than 65536	`400`	closed
EOF in the middle of a frame	—	closed

Path mapping (bserve). The server cuts the path at `?` or `#` and percent-decodes it (`%00` or a bad escape → 400). Any `..` segment → 403. A path ending in `/`, or naming a directory, gets `index.html` added. Symlinks are resolved, and the result MUST stay under the document root (otherwise 403).

7. Why these widths

HTTP/2's frame header is **24/8/8/31** (length/type/flags/stream, plus 1 reserved bit, 9 octets). Its fields are sized for **multiplexing**. Many streams interleave on one connection, so frames must stay small (16 KiB by default) to stop one large body from blocking the others. That makes a 24-bit length (16 MiB ceiling) enough, and a 32-bit length would waste an octet on every frame. The 31-bit stream ID exists because streams are never reused on a long-lived connection and odd/even IDs split the space between client and server. 31 bits (not 32) keeps the ID a positive signed int in languages without unsigned types. The 8-bit type with "ignore unknown" is HTTP/2's extension mechanism, and **we copy it unchanged**.

BHTTP/1 does **not** multiplex. That changes the sums:

- **Length 32, not 24.** No interleaving means no fairness reason to keep frames small, and one frame can carry a whole body if the sender likes. 32 + 8 + 8 + 16 makes an **8-octet** header: it is aligned, parses with two 32-bit loads, and costs 1 octet less than HTTP/2's. The memory risk of a large Length is handled by the receiver, not the field width: DATA is streamed and never buffered whole, unknown frames are discarded in 4 KiB pieces, and a HEADERS frame over 64 KiB is refused.
- **Type 8, Flags 8.** v1 uses 2 types and 1 flag bit. That leaves 254 types and 7 bits for later versions, which is enough because unknowns are skipped or ignored.
- **Req ID 16, not 31.** v1 only needs IDs to be distinct among the requests still waiting for a reply. The field still earns its 2 octets: every response frame says which request it answers, so pipelining, `-v` dumps and desync bugs explain themselves (bcurl rejects a frame for a request it didn't send). It also means a v2 can add multiplexing without changing the header.
- **Header fields.** These are HPACK's first two mechanisms, the static table and length-prefixed literals. We use fixed-width lengths instead of HPACK's prefix varints, so a stranger can decode them by eye from a hexdump. The index needs 1 octet because the table has 10 entries. A name length needs 1 octet because real names are under 40 characters. A value length needs 2 octets because 64 KiB covers long paths and cookies, and anything larger is abuse. We left out HPACK's dynamic table and Huffman coding: both keep state across requests, are hard to debug, and opened the door to CRIME-style and table-exhaustion attacks.

Cost: the request in `HEXDUMP.md` is 63 octets (8 header + 55 block). The same request in HTTP/1.1 text is 86 octets.